Unterstützung durch IntelliJ – Eine kurze Einführung

Einführung in die Programmierung 1

TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?

- Verschachtelte Schleifen
- Komplexere Kombinationen von Schleifen mit Verzweigungen

Was wird in diesem Foliensatz besprochen?

- Unterstützung beim Programmieren durch IntelliJ
- Debugging
- KI-Unterstützung

Was lernen Sie kennen?

- IntelliJ-Unterstützung verstehen und anwenden
- Einfache Debugging-Schritte
- Möglichkeiten für die KI-Unterstützung in IntelliJ

Fehler in Programmen

Fehler bei der Übersetzung

- Werden vom Compiler erkannt
- Keine Übersetzung und daher kein ausführbares Programm

Laufzeitfehler

- Programm kann übersetzt werden
- Beim Ablauf des Programms tritt ein Fehler (z.B. Exception) auf

Logische Fehler

- Programm kann übersetzt werden und läuft bis zum Ende
- Ergebnis bzw. Ausgabe ist falsch

IntelliJ-Fehler (Beispiel)

```
public class Test {

    public static void main(String[] args) {
        int x = 50;
        int y = 20;
        double z = 30;
        System.out.println(x + y)
        int result =  x * z;
        System.out.println(result);

    }
}
```

Problems: File 2 | Project Errors 2 | Server-Side Analysis New | Vulnerable Dependencies

Test.java D:\Unterricht\EP1\SS24\ep1_24s_org\beispiele\Variablen\src 2 problems

- ';' expected :8
- Incompatible types. Found: 'double', required: 'int' :9

Fehler bei der Übersetzung

Fehler und IntelliJ

Fehler bei der Übersetzung	Laufzeitfehler und logische Fehler
• Werden von IntelliJ immer angezeigt	• Manchmal weisen Warnungen darauf hin • Sehr oft können diese aber nicht so einfach erkannt werden • Weitere Maßnahmen (z.B. Debugger verwenden) notwendig

IntelliJ –Warnungen

- Das Programm wird meist funktionieren
- Die Warnungen sollten aber zumindest überprüft werden
- Meist kann der Code angepasst werden und die Warnung verschwindet
- Bei einigen der Beispielprogramme vorhanden, da ein bestimmtes Vorgehen gezeigt werden soll

IntelliJ-Warnungen (Beispiel)

```
public class Test {
    public static void main(String[] args) {
        int a = 5;
        int b = 5;
        int c = 3;
        System.out.println(c / (a - b));
    }
}
```

```
"C:\Program Files\Java\jdk-21\bin\java.exe" --enable-preview "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2023.3.3\lib\idea_rt.jar=55017:C:\
Exception in thread "main" java.lang.ArithmeticException Create breakpoint : / by zero
    at Test.main(Test.java:6)

Process finished with exit code 1
```

Problems: File 2 | Project Errors | Server-Side Analysis New | Vulnerable Dependencies

Test.java D:\Unterricht\EP1\WS24\ep1_24w_org\beispiele\Variablen\src 2 problems

- Division by zero :6
- Result of 'a - b' is always '0' :6

Potentielles Problem zur Laufzeit wird angezeigt
Achtung: Das kann nicht immer so leicht ermittelt werden!

IntelliJ-Warnungen (Beispiel)

```
public class Test {

    public static void main(String[] args) {
        int x = 50;
        int y = 20;
        int z = 30;
        System.out.println(x + y);
        double result = x / y;
        System.out.println(result);

    }
}
```

Das Ergebnis ist wohl nicht das, was erwartet wurde!

```
"C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2023.3.3\lib\idea_rt.jar=61815:C:\Program Files\JetBrains\IntelliJ
70
2.0
```

Problems: File 2 | Project Errors | Server-Side Analysis New | Vulnerable Dependencies

Test.java D:\Unterricht\EP1\SS24\ep1_24s_org\beispiele\Variablen\src 2 problems

- Variable 'z' is never used :7
- 'x / y': integer division in floating-point context :9

Warnung bei if-Anweisung ohne Rumpf

- Beispiel

```
if (x > 20); {
    x = x - 10;
}
```

- Entspricht eigentlich

```
if (x > 20)
    ;
{
    x = x - 10;
}
```

Im if-Zweig steht eine leere Anweisung und der Block danach wird immer ausgeführt

- Auch bei fehlerhaftem else

```
if (x > 20) {
    x = x - 10;
} else; {
    x++;
}
```

Allgemeine Warnungen bei Bedingungen

Intellij erkennt und markiert farblich z.B.	
Bedingungen, die immer wahr / falsch sind	Leere Zweige

```
int x = 10;

/*
Keine Änderung mehr von x
 */

if (x < 10) {
    Sy
} else
    System.out.println(x + 2);
}
```

Condition 'x < 10' is always 'false'

Remove 'if' statement Alt+Umschalt+Eingabe More actions... Alt+Eingabe

Warnungen bei unnötigen Vergleichen

```java
if (check == true) {
    System.out.println(x);
}
```

```java
if (check) {
    System.out.println(x);
}
```

Hat check den Wert true, dann liefert check == true den Wert true
Hat check den Wert false, dann liefert check == true den Wert false
Daher kann direkt check verwendet werden!

Warnungen bei Schleifen (kleine Auswahl)

- Zu viele Iterationen

```
for (int i = 0; i < 10; i--) {
    Loop executes zero or billions of times
}
```

- Update hat keine Auswirkung (Endlosschleife)

```
for (int i = 0; i < 10; i = i * 2) {
    sum += i;
}
```

Result of 'i * 2' is always '0'

Replace with '0' Alt+Umschalt+Eingabe More actions... Alt+Eingabe

- Kein Update der Zählvariable (Endlosschleife)

```
int i = 0;
while (i < 10) {
    sum +=
}
```

Condition 'i < 10' is always 'true'

Simplify 'i < 10' to true Alt+Umschalt+Eingabe More actions... Alt+Eingabe

fori Live-Template

```
fori
```

fori Create iteration loop

Press Strg+Leertaste to see non-imported classes Next Tip

```
for (int i = 0; i < ; i++) {

}
```

```
for (int counter = 0; counter < 100; counter++) {

}
```

Was passiert hier?

```java
public class StarPattern {
    public static void main(String[] args) {
        int n = 2;
        for (int i = -n; i <= n; i++) {
            for (int j = -n; j < n; j++) {
                if (i * i < j * j) {
                    System.out.print("* ");
                } else {
                    System.out.print("  ");
                }
            }
            System.out.println();
        }
    }
}
```

Keine Fehler angezeigt!
Keine Warnungen angezeigt!

Ausgabe, die wir erwarten	Ausgabe, die wir bekommen
* * * * * * * * * * * * * * * * *	 * * * * *

Debugging

Debugger

- Werkzeug zum Diagnostizieren und Auffinden von Fehlern in Programmen

Typische Funktionen eines Debuggers

- Steuerung des Programmablaufs (Haltepunkte, engl. Breakpoints)
- Schrittweise Durchführung von Programmen
- Inspizieren und modifizieren von Variablen

Debugger

File Edit View Navigate Code Refactor Build Run jGRASP Tools Git Window Help Schleifen [D:\Unterricht\EP1\WS21\ep1_21w_org\beispiele\Schleifen] - StarPattern.java

Schleifen › src › StarPattern

StarPattern.java

```
1  public class StarPattern {
2
3      public static void main(String[] args) {   args: []
4          int n = 5;   n: 5
5          for (int i = -n; i <= n; i++) {   i: -5   i: -5
6              for (int j = -n; j <= n; j++) {   n: 5
7                  if (i * i <= j * j) {
8                      System.out.print("* ");
9                  } else {
10                     System.out.print("  ");
11                 }
12             }
```

Debug: StarPattern

Frames: "main"...UNNING; main:6, StarPattern

Variables: args = {String[0]@807} []; n = 5; i = -5

Memory: No classes loaded. Load classes

Breakpoint

Debugger einschalten

Einzelne Programmschritte ausführen

Variablen inspizieren

Werte von Ausdrücken inspizieren

```
8
9    int result = a + b * c;   a: 2   b: 3   c: 5
10
11   }
```

15

Maus über *

```
8
9    int result = a + b * c;   a: 2   b: 3   c: 5
10
11   }
```

17

Maus über +

Live-Beispiel

```java
public class StarPattern {

    public static void main(String[] args) {
        int n = 5;
        for (int i = -n; i <= n; i++) {
            for (int j = -n; j < n; j++) {
                if (i * i < j * j) {
                    System.out.print("* ");
                } else {
                    System.out.print("  ");
                }
            }
            System.out.println();
        }
    }

}
```

Live-Beispiel für einfachen Debugger-Durchlauf

Breakpoints an verschiedenen Stellen setzen

Schrittweiser Ablauf

Variablen inspizieren

Was passiert hier ? – Lösung

```java
public class StarPattern {

    public static void main(String[] args) {
        int n = 2;
        for (int i = -n; i <= n; i++) {
            for (int j = -n; j <= n; j++) {
                if (i * i <= j * j) {
                    System.out.print("* ");
                } else {
                    System.out.print("  ");
                }
            }
            System.out.println();
        }
    }

}
```

```
*       *
* *   * *
* * * * *
* *   * *
*       *
```

KI-Assistent in IntelliJ (kostenpflichtig)

Überblick

- https://www.jetbrains.com/help/idea/ai-assistant.html

Kann z.B. Fehler und Warnungen erklären

- Persönlicher „Tutor“

Ab jetzt werden immer wieder Chats mit der KI in der Vorlesung gezeigt

- Potential aufzeigen (Z.B. Erklärung, Generierung, Fehler ausbessern)
- Probleme aufzeigen (Z.B. Fehler, Code-Qualität, Sicherheit, Copyright)
- Einsatz (kritisch) diskutieren

Beispiel – erster Versuch

Declarations.java | Test.java

```java
public class Test {

    public static void main(String[] args) {
        int x = 50;
        int y = 20;
        int z = 30;
        System.out.println(x + y);
        double result =  x / y;
        System.out.println(result);
    }
}
```

Extract | Surround

- Explain Code
- Suggest Refactoring
- Find Problems
- New Chat Using Selection
- Write Documentation
- Generate Unit Tests
- Generate Code...
- Convert File to Another Language
- Add Your Prompts...

AI Assistant | All Chats | New Chat

Me

Find potential problems in

```java
int x = 50;
int y = 20;
int z = 30;
System.out.println(x + y);
double result =  x / y;
System.out.println(result);
```

AI Assistant

The potential problem in your code lies in this line:

```java
double result =  x / y;
```

Here, `x` and `y` are both `int` (integer) types. Java performs an integer division if both operands are integers. Consequently, it will round the result down to the nearest integer.

In this case, `50 / 20` would normally yield `2.5` if both `x` and `y` were `double` types. But, as `x` and `y` are integers, this results in `2.0`, which is not the accurate value.

So in order to fix this issue, you will either need to cast one of, or both the integers to a `double` before performing the division. Here is the corrected code:

```java
double result =  (double)x / y;
```

This line of code casts `x` to a `double` before the division is performed which means Java will return a `double` representing the exact quotient of `x` and `y`.

Also, it seems that the variable `z` is declared but never used, although this would not cause a problem in compiling or running the program, it is generally good practice to eliminate unused variables.

Share your feedback

Ask AI Assistant

Beispiel – zweiter Versuch

Declarations.java | Test.java

```java
public class Test {

    public static void main(String[] args) {
        int x = 50;
        int y = 20;
        int z = 30;
        System.out.println(x + y);
        double result =  x / y;
        System.out.println(result);
    }
}
```

Extract Surround

AI Assistant | All Chats | New Chat

Me

Find potential problems in

```java
int x = 50;
int y = 20;
int z = 30;
System.out.println(x + y);
double result =  x / y;
System.out.println(result);
```

AI Assistant

The potential issue in this piece of code may be related to the integer division being assigned to a double variable. In this case, it's the statement `double result = x / y;`. Since x and y are integer variables, the division will be an integer division (i.e., the fractional part is truncated). Thus, the result is an integer which is then promoted to a `double`.

The right way to solve this issue in order to get a precise calculation is to explicitly cast x or y (or both) to `double`. Here is the corrected code:

```java
public static void main(String[] args) {
    int x = 50;
    int y = 20;
    int z = 30;
    System.out.println(x + y);
    double result =  (double) x / y;
    System.out.println(result);
}
```

In the corrected code, x is cast to a `double`. This ensures that the division is a floating point division and the result is a double that correctly represents the division.

P.S: variable z is defined, but never used. This unused variable can be considered a minor issue, it's not causing an error but constitutes clutter in the code. This can be cleaned by removing the unused declaration.

Share your feedback

Ask AI Assistant

Ähnliche Erklärung, aber anders formatiert.

Beispiel – dritter Versuch (deutscher Prompt)

Declarations.java | Test.java

```java
public class Test {

    public static void main(String[] args) {
        int x = 50;
        int y = 20;
        int z = 30;
        System.out.println(x + y);
        double result =  x / y;
        System.out.println(result);
    }
}
```

AI Assistant | All Chats | New Chat

Me

Welche Probleme treten im folgenden Programm auf?

```java
int x = 50;
int y = 20;
int z = 30;
System.out.println(x + y);
double result =  x / y;
System.out.println(result);
```

AI Assistant

In Ihrem Java-Programm gibt es mindestens zwei potenzielle Probleme:

1. In der Zeile, in der Sie `x` durch `y` teilen und das Ergebnis auf `result` speichern, wird eine Ganzzahldivision ausgeführt, weil sowohl `x` als auch `y` Ganzzahlen (`int`) sind. Das Ergebnis wird dann zu `double` konvertiert. Wenn Sie jedoch erwarten, dass Ihr Ergebnis eine Fließkommazahl ist, könnte das ein Problem sein, da Ganzzahldivision immer Abrundung auf die nächste Ganzzahl durchführt.
2. Ihre Variable `z` wird deklariert, aber nicht verwendet. Dies führt nicht zu einem Ausführungsfehler, kann jedoch zu Warnungen über ungenutzte Variablen führen, je nachdem, wie Ihre IDE konfiguriert ist.

Auf Lösungen basierende Vorschläge:

1. Um eine Division mit Fließkommazahl zu erzwingen, konvertieren Sie mindestens einen der Operanden (x oder y) zu `double`:

```java
double result = (double)x / y;  //Eine explizite Typkonvertierung wird durchgeführt.
```

2. Überlegen Sie, ob `z` im Code verwendet werden sollte. Wenn es nicht benötigt wird, können Sie die Definition entfernen.